

Exercise 3 — Global User Task Listeners

Goal: wire up three cluster-wide user task listeners — `creating`, `assigning`, `completing` — that fire for every user task in the Office Alpaca process without touching the BPMN.

What you will learn

- The Camunda 8.9 difference between **model-level** and **global** user task listeners.
- The user task lifecycle and which events a listener can react to.
- How to expose a listener as a normal Spring `@JobWorker` (the engine treats it like any other job).
- How to register a global listener via the **Admin UI** in SaaS.
- How to use the *correcting* mechanism inside a `creating` listener to set defaults (e.g., due date).

Prerequisites

1. Exercise 1 complete — the office-alpaca-app starts and connects to your SaaS cluster.
2. Admin access to the SaaS cluster (you need to be able to manage global listeners).

Step 1 — Understand the lifecycle

A native (8.5+) user task moves through:

```
creating → CREATED → assigning → ASSIGNED → completing  
→ COMPLETED  
  
canceling → CANCELED
```



A listener pauses the lifecycle transition until your worker reports completion. Five event types are supported: `creating`, `assigning`, `updating`, `completing`, `canceling`, plus the shorthand `all`.

In this exercise we wire one listener for two events.

Step 2 — Create the handler

Create `AlpacaTaskListener.java` :

```
@JobWorker(type = "alpaca-task-audit", autoComplete = true)
public void onCreating(final ActivatedJob job) {
    var ut = job.getUserTask();           // explore UserTaskProperties:
    key, assignee, dueDate, ...
    //log task details
}
```

Step 3 — Verify the worker is registered

Restart `mvn spring-boot:run` . You should see:

```
Configuring 1 Job worker(s) of bean 'alpacaTaskListeners':
- JobWorkerValue{type='alpaca-task-creating', ...}
```

At this point the workers exist, but the **cluster has not been told to invoke them** yet. That's the global-listener registration in Step 4.

Step 4 — Register the global listeners in Admin

SaaS path — for Self-Managed, use the YAML in `camunda.cluster.global-listeners.user-task` or the Orchestration Cluster REST API. See the reference links.

1. In SaaS, open the **Admin** application for your cluster.
2. Select the **Global User Task Listeners** tab.
3. Click **Create listener** and add the three entries:

Listener ID	Listener type	Event type	Retries	Execution order	Priority
alpaca-audit-create	alpaca-task-audit	Creating	3	Before model-level	50
alpaca-audit-complete	alpaca-task-audit	Creating	3	Before model-level	50

4. Save the listener. Admin applies them immediately to **all running and new** process instances.

Step 5 — Trigger and observe

1. Start a new process instance (`c8 create pi office-alpaca-request ...` or via Tasklist or via Play).
2. Complete the **Upload request documents** task.
3. In your Spring Boot app log you should see something like:

```
taskKey=...   elementId=Task_FacilitiesReview ...
taskKey=...   elementId=Task_CA0Review          ...
```

Step 6 — Demonstrate listener failure handling

Edit `onCreating` to throw `new RuntimeException("simulated failure")` after the audit log. Re-run a process:

- The job fails three times (`retries=3` from Step 4), then the user task remains in `CREATING` and Operate shows an **incident**.
- The user task is *not yet visible in Tasklist* because the lifecycle transition is blocked until the listener succeeds.
- Fix the worker code, redeploy. Operate's incident resolution will retry the listener and the task will appear normally.

This is the key behavioural difference vs an audit aspect tacked onto a worker: a failing listener actually blocks the transition.

Acceptance criteria

- ☐ Three global listeners are visible in the Admin UI.
- ☐ Starting a process emits the `creating`, `completing` audit logs.
- ☐ You demonstrated incident behaviour by making the listener throw, then recovered.

Reference links

- [Global user task listeners concept](#)
- [Configure global listeners](#)
- [Manage listeners in Admin UI](#)
- [User task listener concepts \(lifecycle, blocking behaviour, corrections\)](#)
- [Orchestration Cluster API — Create global task listener](#)